



Message Broker와 RabbitMQ

박연종

2026.09.03.

System Software Lab.

목차

01 Observer Patten

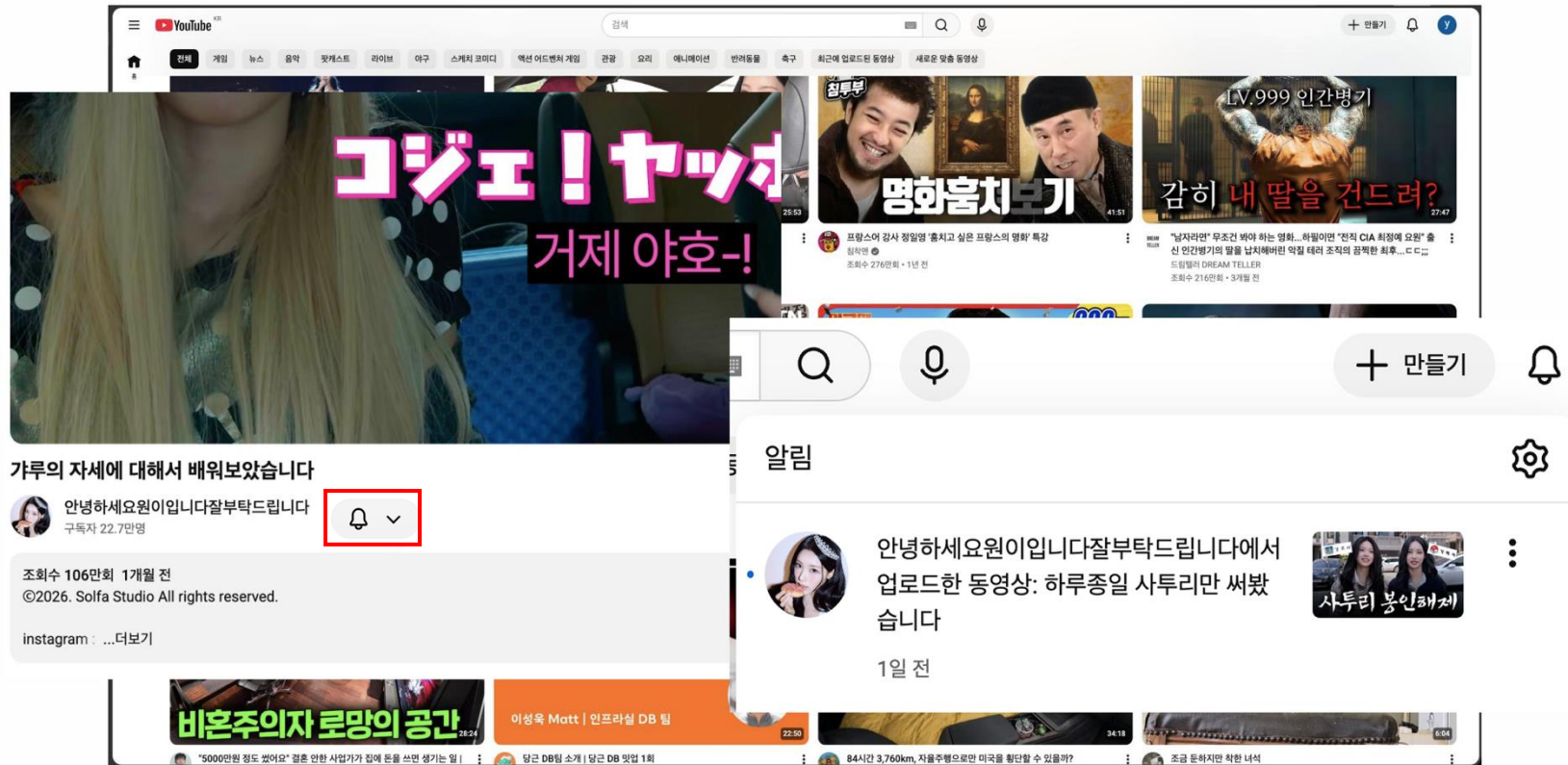
02 Message Broker

03 RabbitMQ

지난 시간에...

Observer Pattern

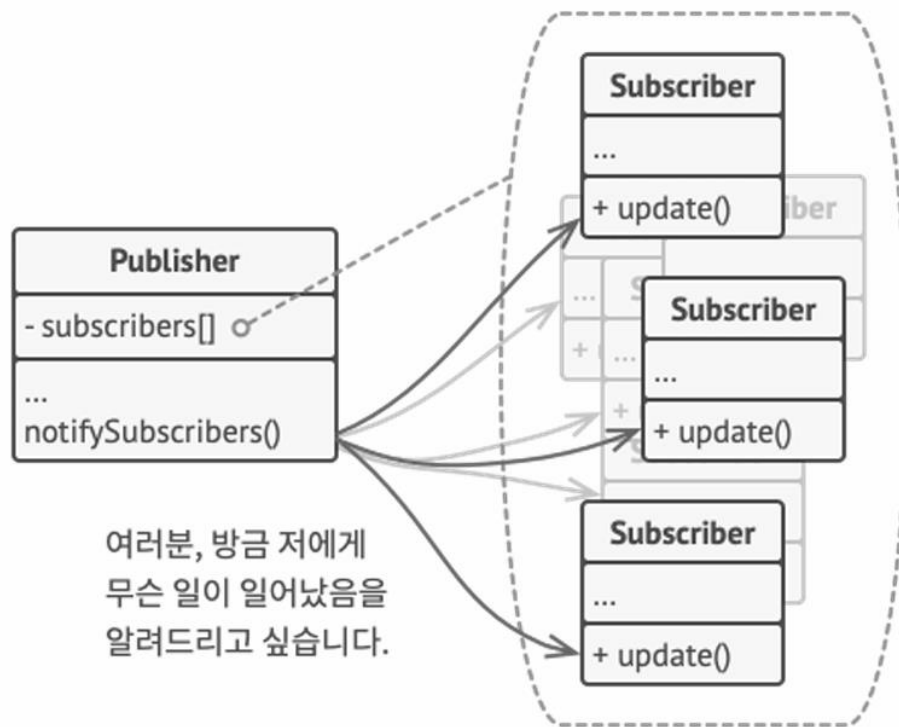
객체 상태가 변경되면 그 객체를 지켜보던 다른 객체들에게
자동으로 알리는 일대다 의존성을 정의하는 디자인 패턴



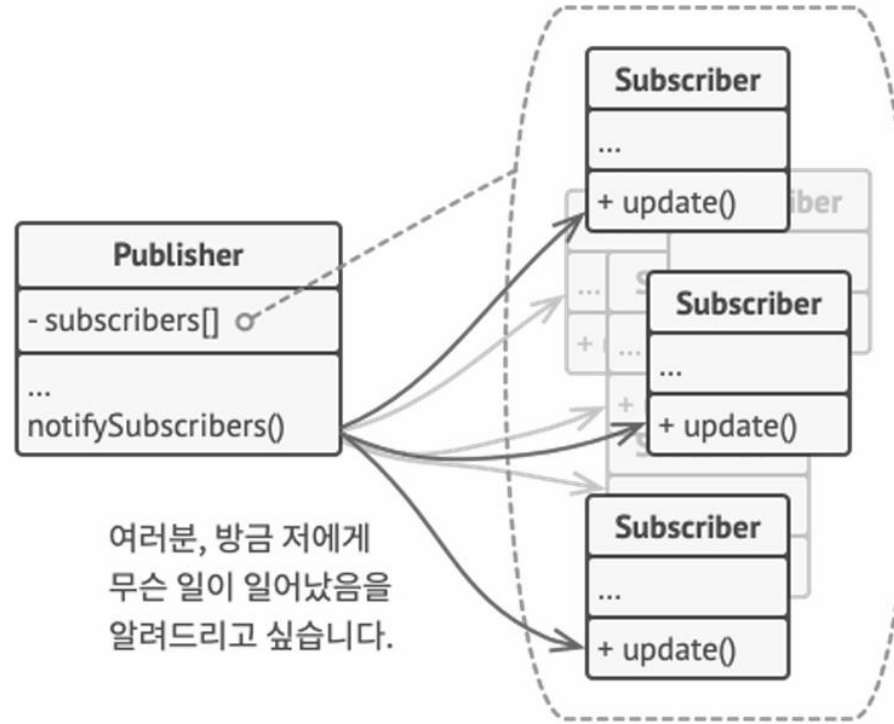
지난 시간에...

Observer Pattern

객체 상태가 변경되면 그 객체를 지켜보던 다른 객체들에게
자동으로 알리는 일대다 의존성을 정의하는 디자인 패턴



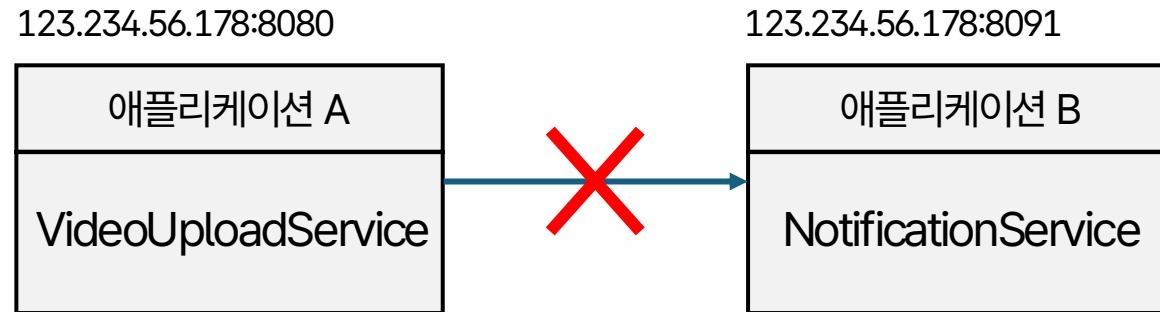
Observer Pattern에서 Subscriber에게 어떻게 알렸나?



Subscriber의 update 메서드를 직접 호출

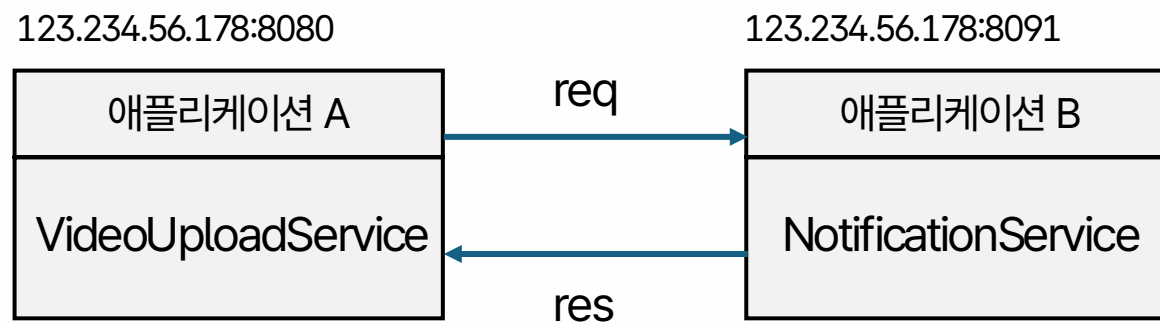
실제 서비스에서는 기능별로 애플리케이션이나 프로세스 분리하는 경우가 많음

Publisher와 Subscriber가 서로 다른 프로그램에 존재한다면?



ApplicationA의 VideoUploadService가 ApplicationB의 NotificationService를 직접 호출할 수 없음

HTTP나 gRPC 요청으로 해결할 수 있지 않나?



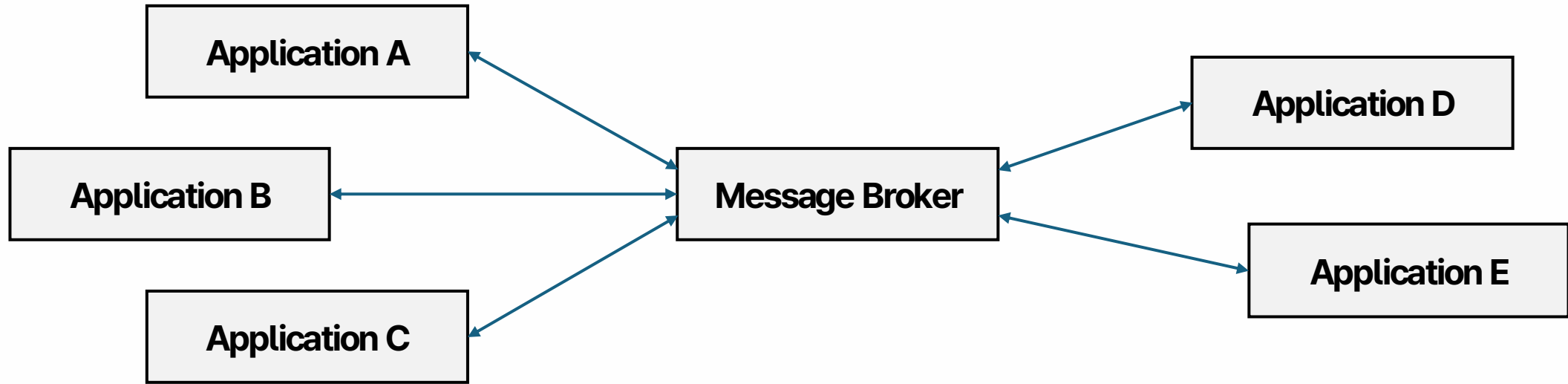
호출 측이 호출을 위해 상대 측 주소와 인터페이스를 알아야 함

요청 결과가 바로 필요한 작업에 적합

일반적인 HTTP 요청은 요청 시점에 상대 측이 처리할 수 있어야 함

상대 측에 장애 발생 시 재시도 로직 등이 필요

Message Broker



Message Broker는 서로 다른 프로그램 사이에서 메시지를 받아 전달하는 중간 소프트웨어

말 그대로 메시지 중개자

Producer와 Consumer가 직접 통신하지 않아도 되며 요청 시점에 바로 처리하지 않아도 됨

RabbitMQ

메시지를 전달하는 Message Broker

Producer

메시지를 생성해 Exchange에게 전달하는 애플리케이션

Exchange

Producer가 보낸 메시지를 어느 Queue로 보낼 지 결정하는 객체

Binding

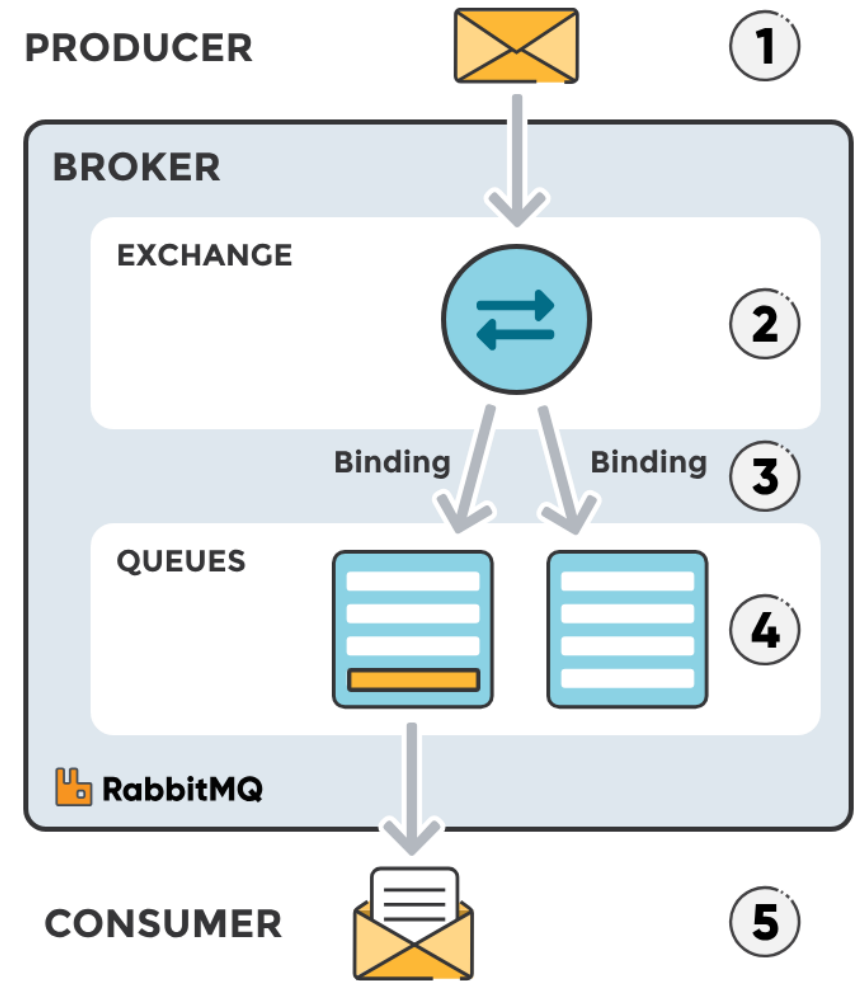
Exchange가 어떤 Queue에 메시지를 전달할 지에 대한 규칙

Queue

Consumer가 처리하기 전까지 메시지가 대기하는 공간

Consumer

Producer의 메시지를 받아 작업을 수행하는 애플리케이션



RabbitMQ Exchange

Direct

바인딩 키와 메시지의 라우팅 키가 **정확히 일치**하는 큐로 메시지를 전달

Topic

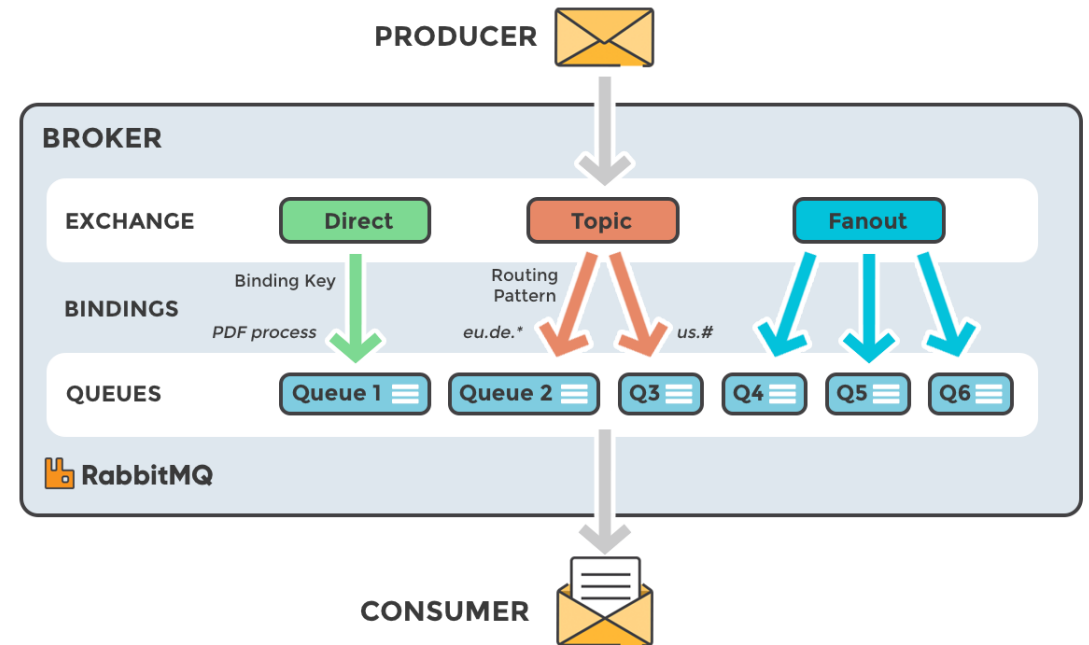
라우팅 키의 바인딩 **패턴 매칭**에 걸리는 큐로 메시지를 전달

Fanout

바인딩된 모든 큐로 메시지를 전달, **브로드캐스트** 개념

Headers

라우팅 키 대신 메시지 헤더와 바인딩 조건이 일치하는 큐로 메시지를 전달



RabbitMQ 설치 및 Spring과 RabbitMQ 연결

implementation 'org.springframework.boot:spring-boot-starter-amqp'

송신 측

```
@Configuration
public class VideoMessagingConfig {
    @Bean
    protected DirectExchange videoExchange() {
        return new
DirectExchange(VideoMessagingConstants.EXCHANGE_NAME);
    }
}
```

수신 측

```
@Configuration
public class VideoMessagingConfig {
    @Bean
    protected DirectExchange videoExchange() {
        return new
DirectExchange(VideoMessagingConstants.EXCHANGE_NAME);
    }

    @Bean
    protected Queue notificationQueue() {
        return QueueBuilder

.durable(VideoMessagingConstants.NOTIFICATION_QUEUE_NAME)
        .build();
    }

    @Bean
    protected Binding videoUploadedBinding(Queue
notificationQueue, DirectExchange videoExchange) {
        return BindingBuilder

        .bind(notificationQueue)
        .to(videoExchange)

        .with(VideoMessagingConstants.VIDEO_UPLOADED_ROUTING_KEY);
    }
}
```

RabbitMQ 설치 및 Spring과 RabbitMQ 연결

송신 측

```
@Service
@RequiredArgsConstructor
public class VideoUploadedEventPublisher {
    private final RabbitTemplate rabbitTemplate;

    public void uploadVideo(VideoUploadedEvent event) {
        rabbitTemplate

        .convertAndSend(VideoMessagingConstants.EXCHANGE_NAME,

        VideoMessagingConstants.VIDEO_UPLOADED_ROUTING_KEY,
        event);
    }
}
```

```
@Service
@RequiredArgsConstructor
public class VideoUploadService {
    private final VideoUploadedEventPublisher eventPublisher;

    public void upload(Long channelId, Long videoId, String
channelName, String videoTitle) {
        VideoUploadedEvent event =
            new VideoUploadedEvent(
                channelId,
                videoId,
                channelName,
                videoTitle);
        eventPublisher.uploadVideo(event);
    }
}
```

수신

```
@Component
@RequiredArgsConstructor
public class VideoUploadedEventListener {
    private final NotificationService notificationService;

    @RabbitListener(queues =
VideoMessagingConstants.NOTIFICATION_QUEUE_NAME)
    public void receive(VideoUploadedEvent event) {
        notificationService.notifyVideoUploaded(event);
    }
}
```

RabbitMQ 설치 및 Spring과 RabbitMQ 연결

RabbitMQTM

RabbitMQ 4.3.5

Erlang 27.3.4.17

Overview

Connections

Channels

Exchanges

Queues and Streams

Admin

▼ All exchanges (9)

Pagination

Page 1 of 1 - Filter: ☐ Regex ?

Virtual host	Name	Type	Features	Message rate in	Message rate out	+/-
/	(AMQP default)	direct	D			
/	amq.direct	direct	D			
/	amq.fanout	fanout	D			
/	amq.headers	headers	D			
/	amq.match	headers	D			
/	amq.rabbitmq.log	topic	D I			
/	amq.rabbitmq.trace	topic	D I			
/	amq.topic	topic	D			
/	video.exchange	direct	D			

Overview

Messages

Message rates

Virtual host	Name	Type	Features	State	Ready	Unacked	Total	incoming	deliver / get	ack
/	notification.queue	classic	D Args	running	0	0	0			

► Add a new exchange

메시지는 RabbitMQ 안에서 어떻게 움직이나?

POST http://localhost:8091/videos

Docs Params Authorization Headers 9 Body Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary

```
1 {
2   "channelId": 1,
3   "videoId": 1,
4   "channelName": "코딩애플",
5   "videoTitle": "크롬이 자동으로 설치하는 2GB짜리 파일"
6 }
```

세 명의 유튜버가 각자 한 개의 영상을 업로드

Overview					Messages			Message rates		
Virtual host	Name	Type	Features	State	Ready	Unacked	Total	incoming	deliver / get	ack
/	notification.queue	classic	D Args	running	3	0	3	0.00/s		

메시지는 RabbitMQ 안에서 어떻게 움직이나?

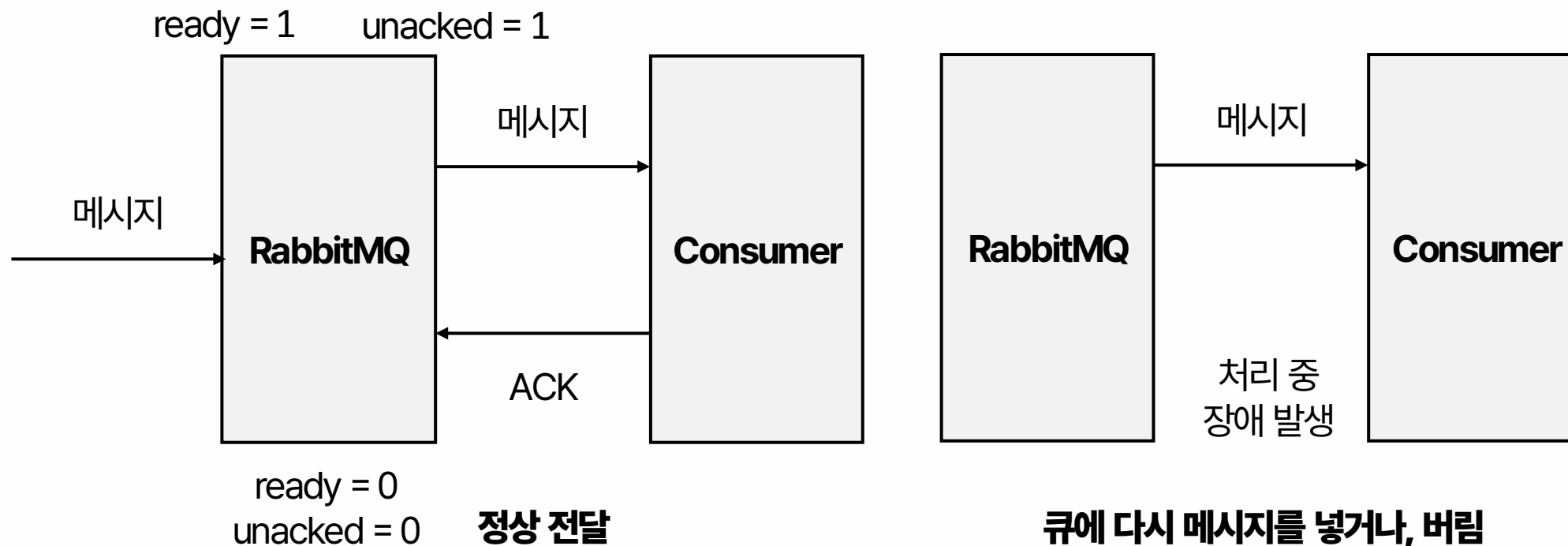
[알림] 코딩애플 채널에 '크롬이 자동으로 설치하는 2GB짜리 파일' 영상이 업로드되었습니다.

[알림] 전자정부 표준프레임워크 센터 채널에 '2025 전자정부 표준프레임워크 신규버전 발표회' 영상이 업로드되었습니다.

[알림] ITSub잇섭 채널에 '충격) USB를 천천히 끼우면 속도가 1/10으로 줄어든다고?' 영상이 업로드되었습니다.

Overview					Messages			Message rates		
Virtual host	Name	Type	Features	State	Ready	Unacked	Total	incoming	deliver / get	ack
/	notification.queue	classic	D Args	running	0	0	0	0.00/s	0.00/s	0.00/s

Consumer가 메시지를 받은 후 오류가 발생하면?



RabbitMQ의 ACK는 TCP의 ACK에서 영감을 받음

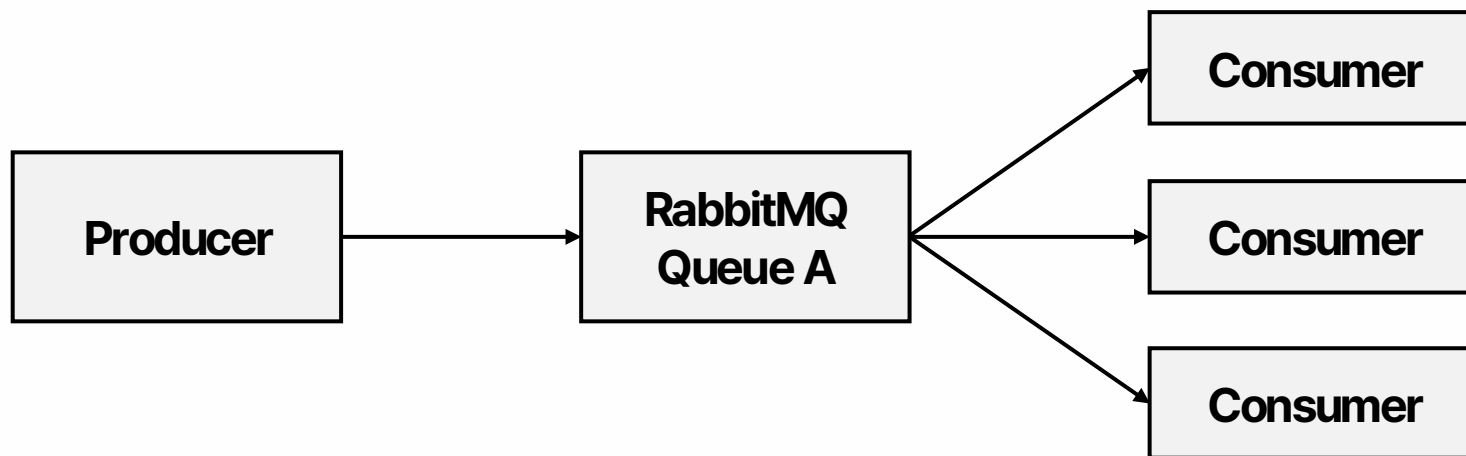
TCP의 ACK는 상대가 잘 받았는지를, RabbitMQ의 ACK는 상대가 받아 잘 처리했는지를 나타냄

큐에 병목이 발생하면?

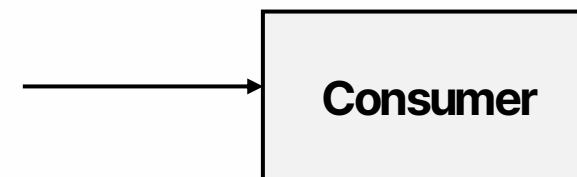


초당 100개의 메시지를 전송 초당 30개의 메시지를 처리

매초 70개의 메시지가 큐에 누적

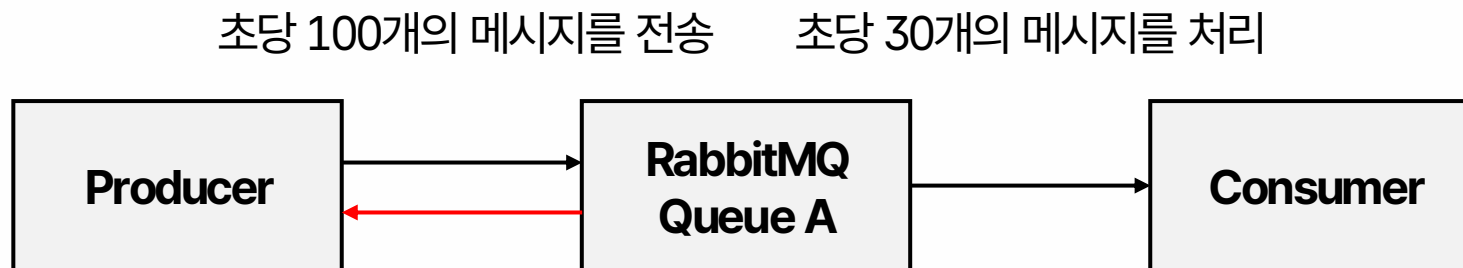


초당 30개의 메시지를 세 Consumer가 처리



초당 100개의 메시지를 처리

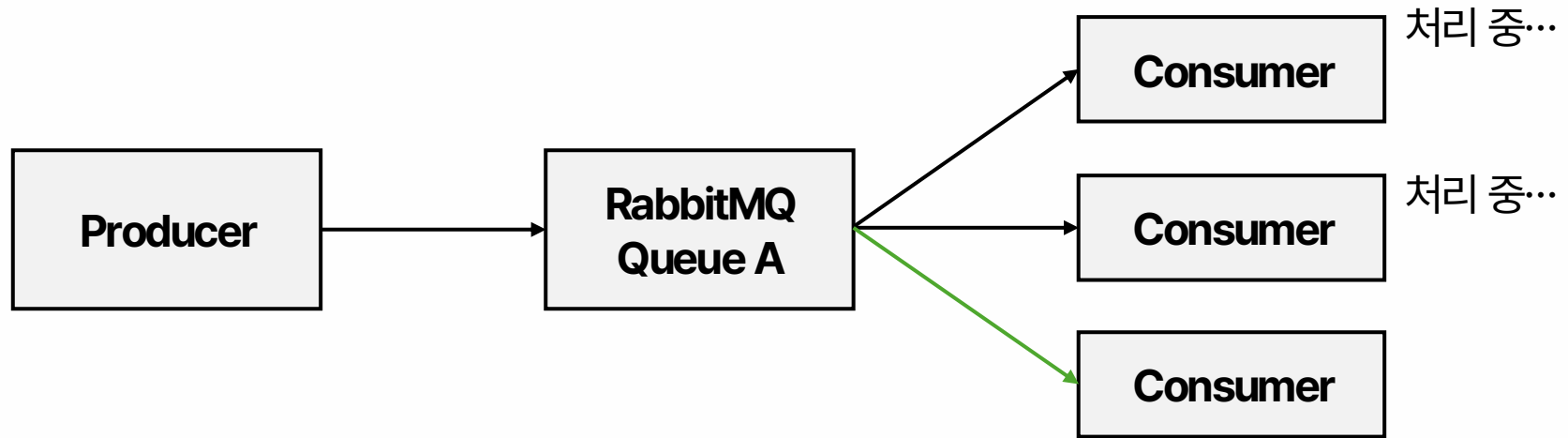
큐에 병목이 발생하면?



큐가 발행 속도를 따라가지 못하면 RabbitMQ가 발행 속도를 자동으로 낮추어 Producer가 느리게 발행하도록 조절

네트워크 혼잡 상태일 때 송신 측이 전송하는 패킷의 수를 조절하는 TCP의 혼잡 제어와 유사

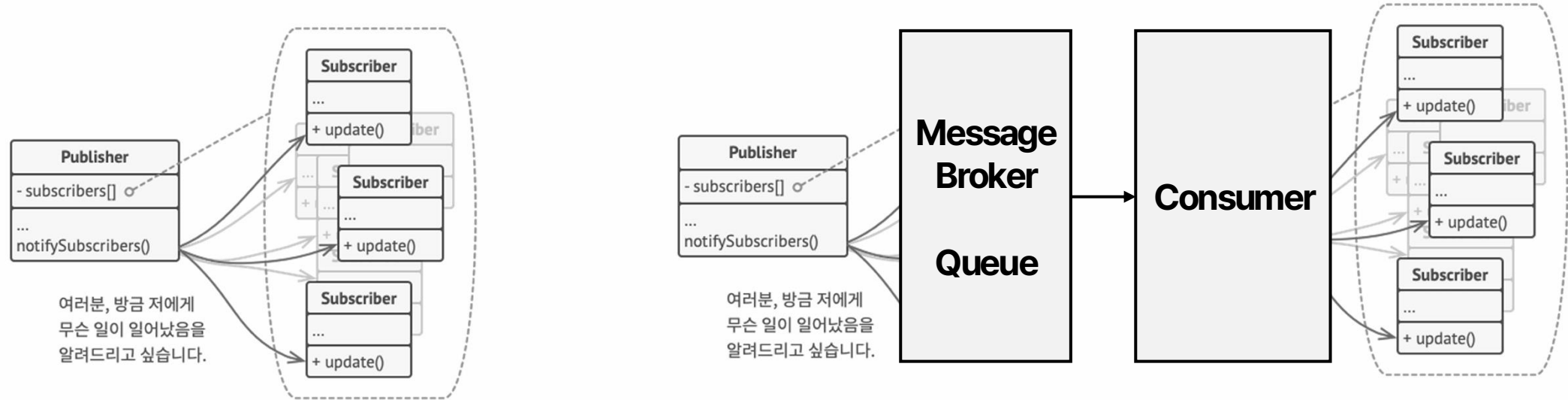
큐에 병목이 발생하면?



prefetch를 적절하게 조절하여 각 Consumer가 동시에 받을 수 있는 메시지의 수를 지정

수신 측이 받을 수 있는만큼 메시지를 전송하는 TCP의 흐름 제어와 유사

Observer Pattern과 비교



Observer에서는 객체가 다른 객체에게 직접 알림

Message Broker를 두어 서로 다른 애플리케이션이 비동기적으로 통신할 수 있음

다음 시간에...

과거 메시지를 다시 확인하고 싶다면?

메시지를 보내고 잘 처리되면 삭제하는 것이 아니라, 장기 보관이 필요하다면?

메시지를 현재 어디까지 읽었는지 확인하고 싶다면?



들어 주셔서 감사합니다

박연종	E-mail	park@duck.com
	LinkedIn	yeonjong-park
	Instagram	yeonjong.park
	GitHub	patulus